Instructions from T1, T2, and T3 arrive in the following order.

R1(A); R2(A); R3(B); R2(C); R2(B); W1(A); W1(C); C1; C3; W2(B); C2;

What is the final *schedule* if the *timestamp ordering protocol* is implemented by CC Manager? Assume that Timestamp (T1,T2,T3)=(1,2,3)

1 R1(A) -> R1(A) = 1 > W-Timestamp(A) = 0, maka R1(A) valid R2(A) -> R2(A) = 2 > W-Timestamp(A) = 1, maka R2(A) valid R3(A) -> R3(B) = 3 > W-Timestamp(B) = 0, maka R3(B) valid R2(C) -> R2(C) = 2 > W-Timestamp(C) = 2, maka R2(C) valid R2(B) -> R2(B) = 2 > W-Timestamp(B) = B, maka R2(B) valid (R-Timestamp(B) tetap 3) W1(A) -> W1(A) = 1 < R-Timestamp(A) = 2. Maka W1(A) tidak valid dan T1 di rollback W1(C) (skip karena T1 abort) C1 (skip karena T1 abort) C3 W2(B) -> W2(B) = 2 < R-Timestamp(B) = 3, maka W2(B) tidak valid dan T2 di rollback C2 (skip karena T2 abort)
Schedule akhir : R1(A); R2(A); R3(B); R2(C); R2(B); A1; C3; A2;

Periksalah apakah schedule

S: R1(X); W2(X); W2(Y); W3(Y); W1(Y); C1; C2; C3;

dapat dihasilkan dengan menggunakan ***Validation-based Protocol***. Timestamp transaksi Ti adalah i dan sebelum S dieksekusi timestamp semua item data adalah 0. **Jelaskan** jawaban Anda.

2

Ya, schedule dapat dihasilkan dengan menggunakan *Validation-based Protocol*.

Transaksi	**Read set**	**Write set**
T1	{X}	{Y}
T2	{}	{X, Y}
T3	{}	{Y}

Validasi terjadi saat commit, sehingga TS(T1) < TS(T2) < TS(T3).

Uji validasi per transaksi:

1. T1: tidak ada Ti dengan TS lebih kecil, jadi lolos. .tetapi kondisi 2 terpenuhi karena write set T1 {Y} ∩ read set T2 {} = ∅, jadi lolos.

2. T3: terhadap T1 dan T2, kondisi 1 gagal, tetapi kondisi 2 terpenuhi karena read set T3 kosong, jadi lolos.

Dengan demikian, semua transaksi lolos validasi dan commit tanpa abort, sehingga S dapat dihasilkan. Schedule serial yang ekivalen adalah T1 → T2 → T3

Berikut ini adalah urutan kedatangan operasi-operasi dari 4 buah transaksi yang berjalan secara konkuren ke *Transaction Manager*. *Timestamp* transaksi *Ti* adalah *i* dan sebelum S dieksekusi *timestamp* semua item data adalah 0.

R1(A); R2(B); W1(C); R3(D); R4(E); W3(B); W2(C); W4(A); W1(D); C1; C2; C3; C4;

Apabila protokol konkurensi yang digunakan adalah *timestamp ordering protocol*, **tuliskan *schedule* yang dihasilkan** dari eksekusi urutan operasi di atas. *Schedule* harus memuat eksekusi operasi (R atau W atau C) dan A apabila ada sebuah transaksi yang harus *rollback*. Pada saat sebuah transaksi *rollback* semua instruksi transaksi tersebut hingga saat terjadi *rollback* akan diprioritaskan untuk dieksekusi kembali. **Jelaskan** jawaban Anda.

3

R1(A) = 1 > W-Timestamp(A) = 0, maka R1(A) lolos dan R-Timestamp(A) = 1
R2(B) = 2 > W-Timestamp(B) = 0, maka R2(B) lolos dan R-Timestamp(B) = 2
W1(C) = 1 > R-Timestamp(C) = 0 dan W-Timestamp(C) = 0, maka W1(C) lolos dan W-Timestamp(C) = 1
R3(D) = 3 > W-Timestamp(D) = 0, maka R3(D) lolos dan R-Timestamp(D) = 3
R4(E) = 4 > W-Timestamp(E) = 0, maka R4(E) lolos dan R-TImestamp(E) = 4
W3(B) = 3 > W-Timestamp(B) = 0 dan R-Timestamp(B) = 2, maka W3(B) lolos dan W-Timestamp(B) = 3
W2(C) = 2 > W-Timestamp(C) = 1 dan R-Timestamp(C) = 0, maka W2(C) lolos dan W-Timestamp(C) = 2
W4(A) = 4 > W-Timestamp(A) = 0 dan R-Timestamp(A) = 1, maka W4(A) lolos dan W-Timestamp(A) = 4
W1(D) = 1 < R-Timestamp(D) = 3, maka W1(D) tidak lolos dan T1 abort (A)
Maka, restart T1 dan berikan timestamp baru = timestamp terbesar + 1 = 4 + 1 = 5
Lakukan ulang instruksi T1 = R1(A), W1(C), W1(D). Karena timestamp T1 = 5, maka semuanya lolos dan instruksi dilakukan
Selanjutnya, transaction manager akan melakukan C1; C2; C3; C4 untuk commit semua transaksi

Maka, schedule

R1(A); R2(B); W1(C); R3(D); R4(E); W3(B); W2(C); W4(A); A1; R1(A); W1(C); W1(D); C1; rC2; C3; C4;

Periksalah apakah schedule *S: R1(X); W2(X); W2(Y); W3(Y); W1(Y); C1; C2; C3;* dapat dihasilkan dengan menggunakan protokol-protokol berikut ini. *Timestamp* transaksi *Ti* adalah *i* dan sebelum S dieksekusi *timestamp* semua item data adalah 0. **Jelaskan** jawaban Anda.

1. Timestamp ordering
2. Timestamp ordering with Thomas' Write Rule
3. Validation based

4
1. Timestamp Ordering tidak dapat dihasilkan menggunakan protokol timestamp ordering karena gagal di W1(Y) hal ini disebabkan timestamp yang masuk merupakan dari T1 sedangkan yang terakhir dari T3 sehingga schedule di-abort. 2. Timestamp ordering with Thomas' Write Rule berhasil karena memperbolehkan operasi W1(Y) tanpa harus membatalkan T1 lebih diabaikan karena salah satu modifikasi syarat dari yang basic adalah ketika Wi(Q) < Wj(Q) Wi bisa diabaikan atau tetap lanjut karena sudah diwakilkan oleh Wj. 3. Timestamp validation based juga berhasil karena memenuhi 3 syarat yang diminta protokolnya dari read phase, validation, hingga write phase. Untuk ini, ketika validatipn phase T1 otomatis lolos, lalu T2 terhadap T1 masuk ke syarat kedua dan memenuhinya yaitu tidak ada W1(Q) yang beririsan dengan R2(Q). Terakhir T3 terhadap T1 dan T2 menggunakan syarat kedua juga yang memenuhi syarat bahwa tidak ada W1(Q) yang beririsan dengan R3(Q) serta W2(Q) yang beririsan degan R3(Q).